NAMA : RAHMAT FADILLAH PUTRA

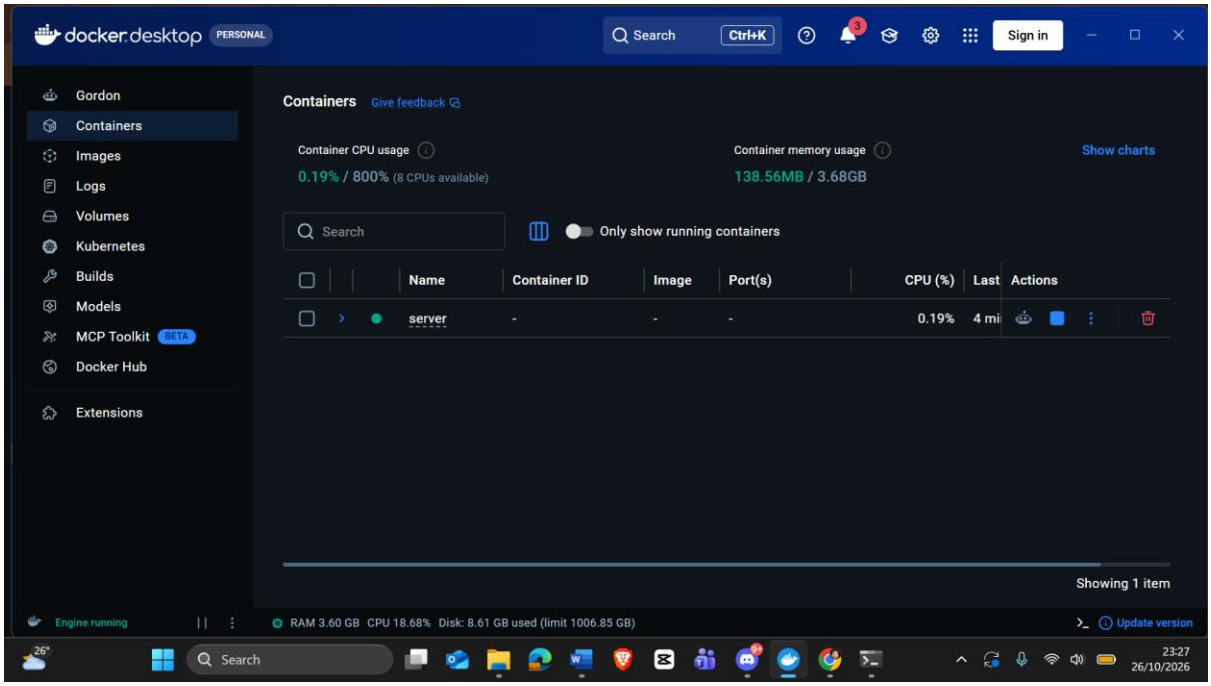
NPM : 25782079

LAB : 7

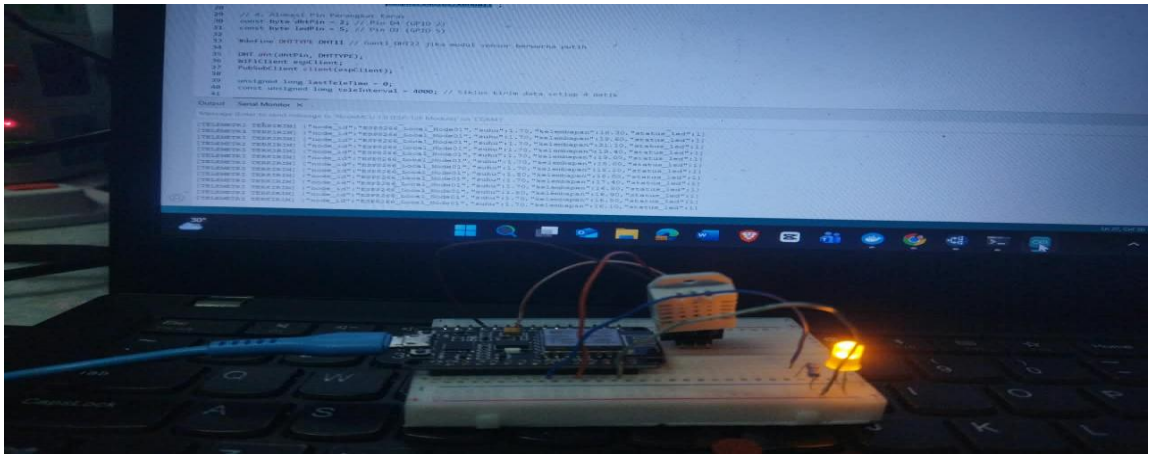
7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

```
E:\server>tree
Folder PATH listing for volume DATA
Volume serial number is CAD5-2AEE
E:.
├───mosquitto
│   ├───config
│   ├───data
│   └───log
└───nodered
    ├───lib
    │   └───flows
    └───node_modules

E:\server>
```



7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

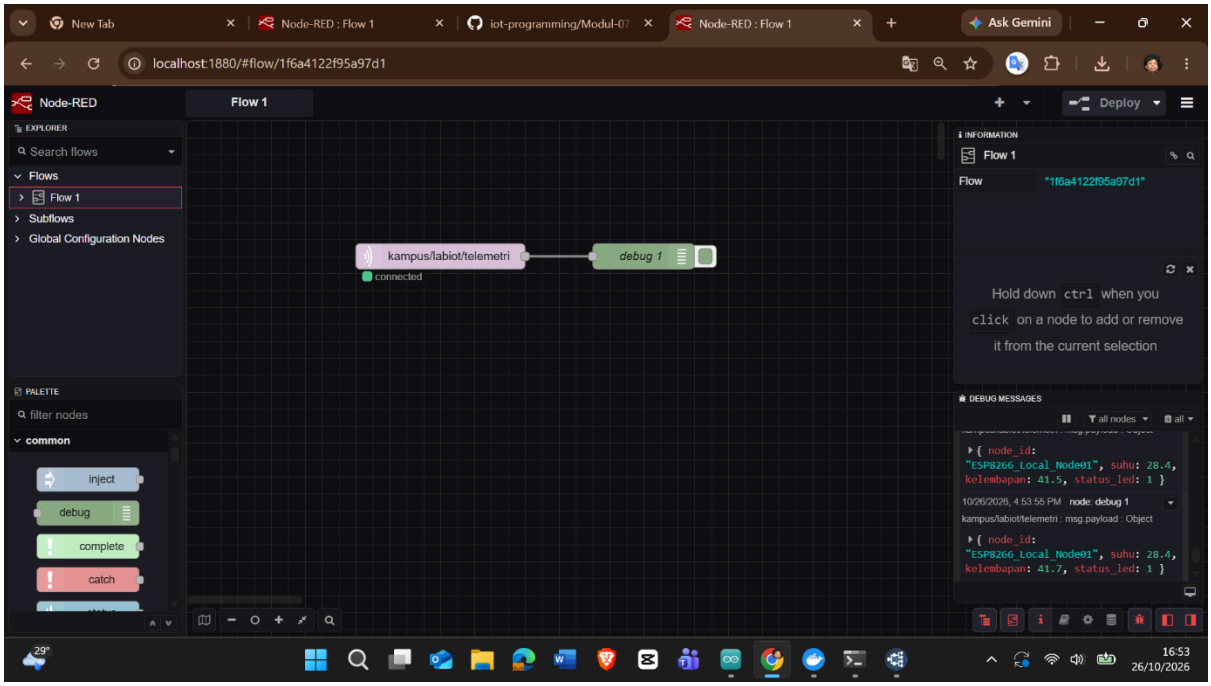


Output Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

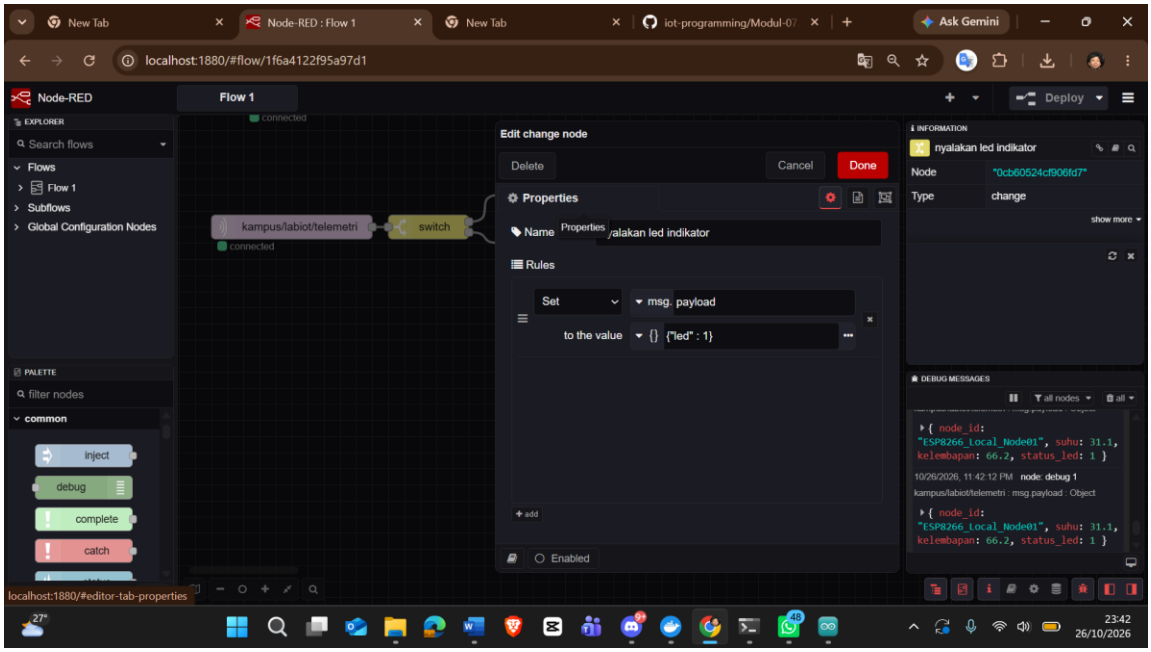
```
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":41.60,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":41.80,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":41.60,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":41.60,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":41.70,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":41.70,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.50,"kelembapan":41.50,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.50,"kelembapan":41.60,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.50,"kelembapan":41.60,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.50,"kelembapan":41.40,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.50,"kelembapan":41.20,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.50,"kelembapan":41.30,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.50,"kelembapan":41.10,"status_led":1}
```

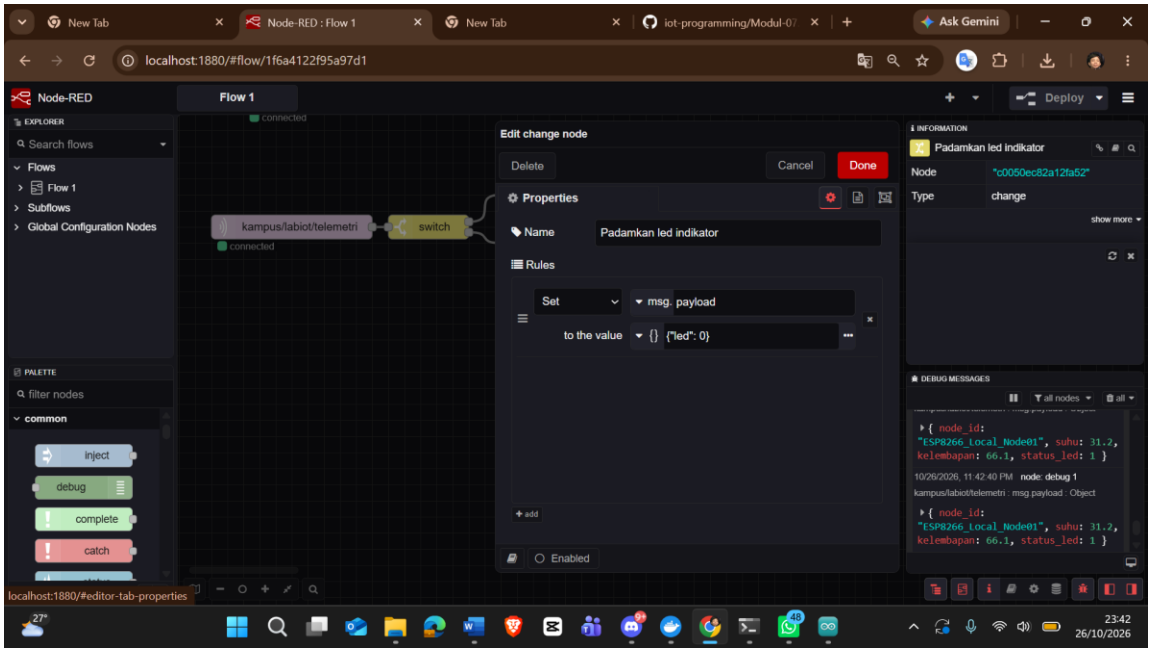
7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

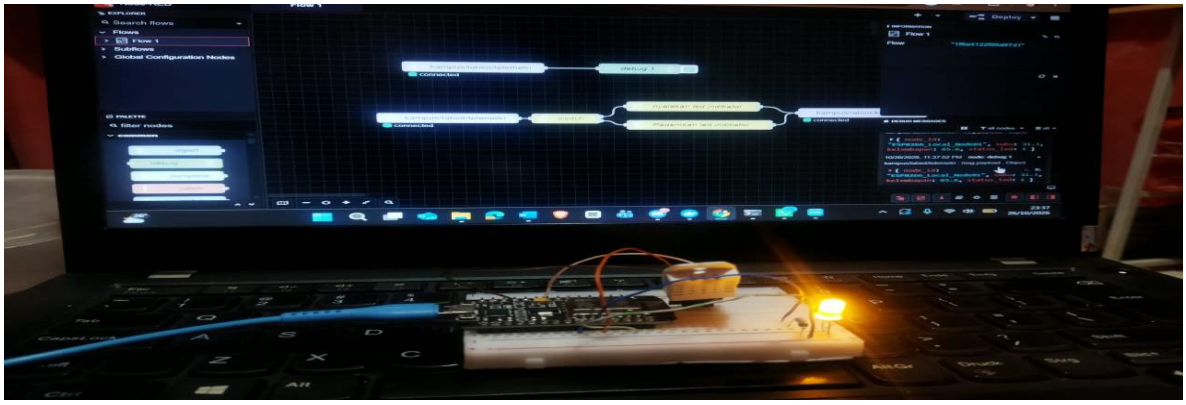


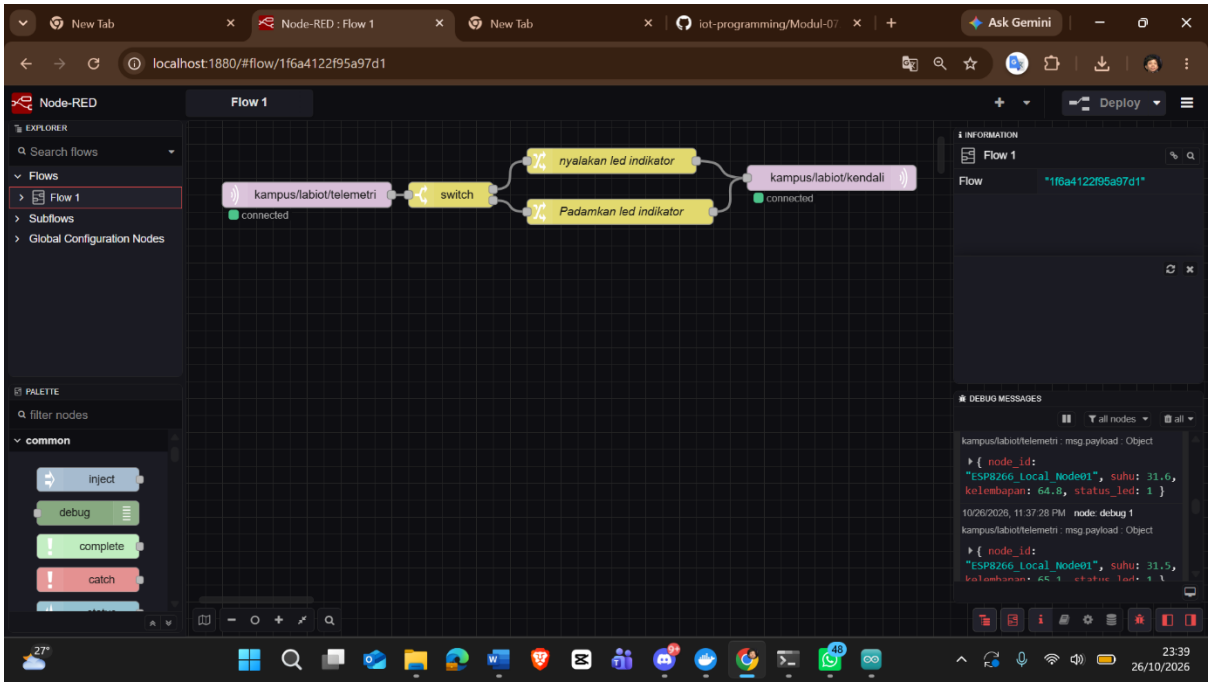
7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

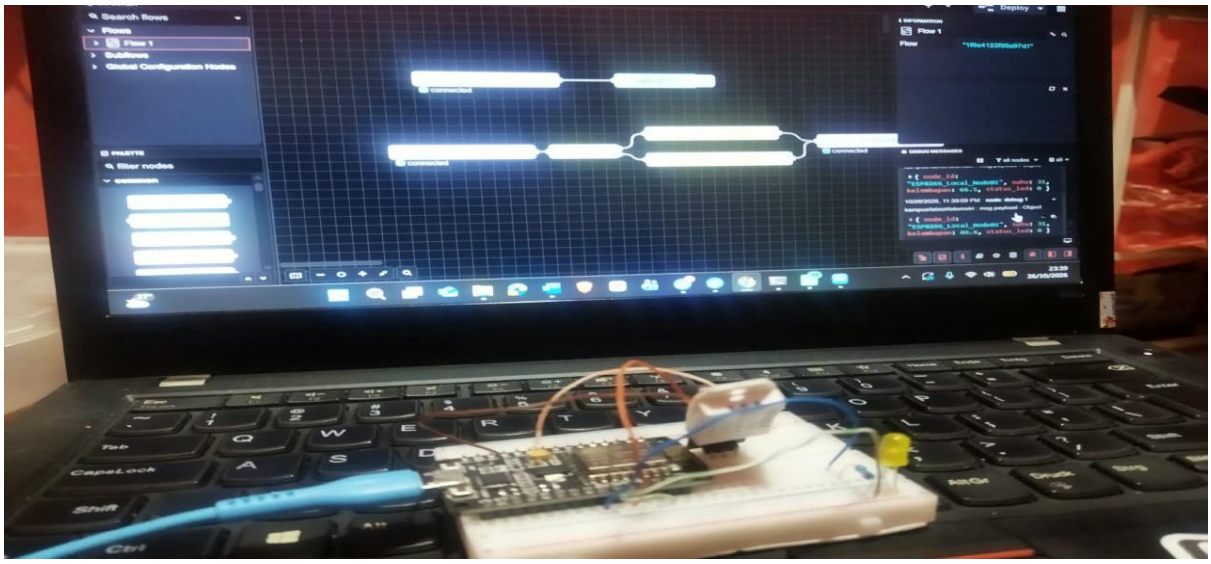

nyalakan led indikator
kampus/labiot/telemetri
switch
kampus/labiot/kendali
connected
Padamkan led indikator
connected

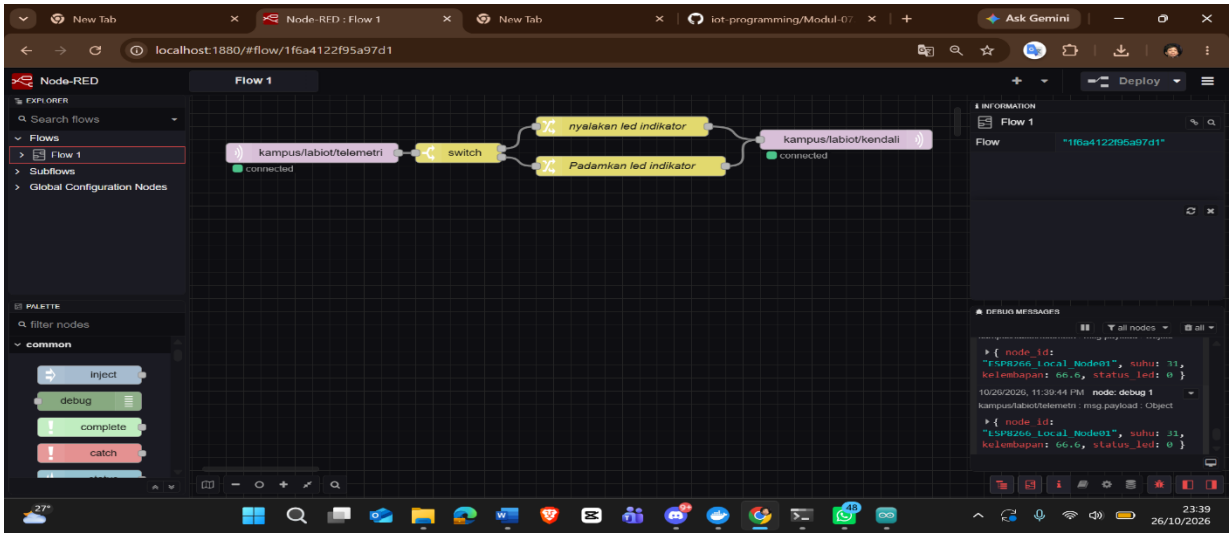

Node-RED : Flow 1
localhost:1880/#flow/1f6a4122f95a97d1
Node-RED
Flow 1
Deploy
Edit change node
Delete
Cancel
Done
Properties
Name
yalakan led indikator
Rules
Set
msg. payload
to the value
{"led" : 1}
Enabled
INFORMATION
nyalakan led indikator
Node
"0cb60524cf906fd7"
Type
change
DEBUG MESSAGES
kampus/labiot/telemetri
switch
inject
debug
complete
catch
common
filter nodes
PALETTE


Node-RED : Flow 1
localhost:1880/#flow/1f6a4122f95a97d1
Node-RED
Flow 1
Deploy
Edit change node
Delete
Cancel
Done
Properties
Name
Padamkan led indikator
Rules
Set
msg. payload
to the value
{"led": 0}
Enabled
INFORMATION
Padamkan led indikator
Node
"c0050ec82a12fa52"
Type
change
DEBUG MESSAGES
kampus/labiot/telemetri
switch
inject
debug
complete
catch
common
filter nodes
PALETTE




New Tab
Node-RED : Flow 1
New Tab
iot-programming/Modul-07
Ask Gemini
localhost:1880/#flow/1f6a4122f95a97d1
Node-RED
Flow 1
Deploy
EXPLORER
Search flows
Flows
Flow 1
Subflows
Global Configuration Nodes
kampus/labiot/telemetri
connected
switch
nyalakan led indikator
Padamkan led indikator
kampus/labiot/kendali
connected
INFORMATION
Flow 1
Flow
"1f6a4122f95a97d1"
PALETTE
filter nodes
common
inject
debug
complete
catch
DEBUG MESSAGES
all nodes
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 31.6, kelembapan: 64.8, status_led: 1 }
10/26/2026, 11:37:28 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 31.5,
27°
23:39
26/10/2026





7.6 Latihan

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks mqtt, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.
  i. **Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?**

JAWAB : Setiap kontainer Docker berjalan dalam lingkungan jaringan yang terisolasi (*network namespace*) dengan *network stack* miliknya sendiri. Ketika Node-RED berjalan di dalam kontainer dan mengakses alamat `localhost` (atau `127.0.0.1`), ia merujuk ke loopback interface kontainer itu sendiri — bukan ke host laptop, dan bukan ke kontainer lain.

Kontainer `node-red` hanya menjalankan proses aplikasi Node-RED di port 1880. Tidak ada proses Mosquitto Broker yang mendengarkan di port 1883 di dalam kontainer `node-red`. Oleh karena itu, ketika Node-RED mencoba membuka koneksi TCP ke `localhost:1883`, tidak ada layanan yang menerima koneksi tersebut, sehingga sistem operasi kontainer mengembalikan respons Connection Refused (ECONNREFUSED).

ii. **Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!**

Jawaban:

Docker Compose secara otomatis menyediakan dua mekanisme yang memungkinkan resolusi nama antar-kontainer:

A. Docker Bridge Network

Ketika perintah `docker compose up` dijalankan, Docker secara otomatis membuat sebuah jaringan bridge virtual khusus untuk proyek tersebut (misalnya `iot-server_default`). Semua kontainer yang didefinisikan dalam `docker-compose.yml` secara otomatis terhubung ke jaringan bridge yang sama ini. Setiap kontainer mendapatkan alamat IP virtual internal yang unik dalam subnet jaringan bridge tersebut (misalnya `172.18.0.2` untuk Mosquitto dan `172.18.0.3` untuk Node-RED).

Jaringan bridge ini berfungsi seperti switch virtual yang memungkinkan semua kontainer di dalamnya berkomunikasi satu sama lain melalui protokol TCP/IP tanpa perlu melewati jaringan eksternal host.

B. Docker Embedded DNS Server (127.0.0.11)

Docker menjalankan sebuah DNS server internal yang tertanam (*embedded*) pada alamat `127.0.0.11` di setiap kontainer. DNS server ini secara otomatis mendaftarkan nama layanan (service name) yang tertulis di `docker-compose.yml` sebagai *DNS record* yang memetakan ke alamat IP internal kontainer terkait.

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
  Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP `192.168.43.102`, sedangkan laptop memiliki alamat IP `192.168.1.15`.
  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

  Jawaban:

  Kegagalan koneksi terjadi karena NodeMCU dan laptop berada di subnet jaringan yang berbeda.

  Kedua perangkat berada di dua jaringan (*subnet*) yang berbeda secara logis:

- NodeMCU terhubung ke jaringan dengan prefix `192.168.43.x` (kemungkinan besar *Mobile Hotspot* dari ponsel).
- Laptop terhubung ke jaringan dengan prefix `192.168.1.x` (kemungkinan besar router Wi-Fi rumah/kampus).

  Dalam arsitektur TCP/IP, perangkat pada subnet yang berbeda tidak dapat berkomunikasi secara langsung tanpa perantara router yang menghubungkan kedua subnet tersebut. Ketika NodeMCU mencoba mengirim paket TCP SYN ke `192.168.1.15:1883`, paket tersebut tidak dapat mencapai laptop karena:

1. NodeMCU menyadari bahwa alamat tujuan (`192.168.1.15`) bukan bagian dari subnet lokalnya (`192.168.43.0/24`).

2. NodeMCU akan mencoba mengirim paket ke *default gateway*-nya, tetapi tidak ada router yang mengetahui rute menuju subnet `192.168.1.0/24` dari subnet `192.168.43.0/24`.
3. Paket di-*drop* dan koneksi MQTT gagal dengan status timeout atau connection refused.

ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

Jawaban:

Mahasiswa harus memastikan laptop dan NodeMCU ESP8266 terhubung ke satu jaringan Wi-Fi yang sama sehingga keduanya mendapatkan alamat IP dalam subnet yang identik. Berikut langkah-langkah teknisnya:

Solusi Utama (Direkomendasikan): Gunakan Mobile Hotspot Ponsel sebagai Satu Titik Akses Bersama

1. Aktifkan fitur *Mobile Hotspot / Tethering* pada ponsel Android atau iPhone.
2. Hubungkan laptop ke jaringan Wi-Fi hotspot tersebut (putuskan koneksi dari router kampus/rumah terlebih dahulu).
3. Hubungkan NodeMCU ESP8266 ke SSID hotspot yang sama melalui kode program Arduino:

```
4. const char* ssid     = "NamaHotspotPonsel";
   const char* password = "PasswordHotspot";
```

5. Verifikasi subnet: Setelah kedua perangkat terhubung, periksa alamat IP masing-masing:
   - Laptop: Jalankan `ipconfig` (Windows) → misal mendapat `192.168.43.15`
   - NodeMCU: Lihat di Serial Monitor → misal mendapat `192.168.43.102`
   - Keduanya kini berada di subnet yang sama: `192.168.43.0/24` ☑
6. Perbarui alamat broker di kode Arduino sesuai IP laptop yang baru:

```
const char* mqttServer = "192.168.43.15";
```

- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**
  Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0∘C (misal: 31.05∘C→30.95∘C→31.02∘C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
  Rancang modifikasi alur Node-RED menggunakan **`function node`** berbahasa JavaScript yang menerapkan logika **Histeresis**:
  - LED baru menyala jika suhu >31.5∘C (`msg.payload = {"led": 1}`).
  - LED baru padam jika suhu mendingin hingga <30.0∘C (`msg.payload = {"led": 0}`).

    Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

```
// ============================================================
// FILTER HISTERESIS - KENDALI LED DENGAN DEADBAND
// ============================================================
// Ambang atas: LED menyala jika suhu > 31.5°C
// Ambang bawah: LED padam jika suhu < 30.0°C
// Zona deadband (30.0 - 31.5): status LED tidak berubah
// ============================================================

var upperThreshold = 31.5;  // Batas atas untuk menyalakan LED
var lowerThreshold = 30.0;  // Batas bawah untuk memadamkan LED

// Ambil nilai suhu dari payload telemetri
var suhu = msg.payload.suhu;

// Ambil status LED terakhir dari flow context (default: 0 = padam)
var lastLedState = flow.get("lastLedState") || 0;

// Evaluasi logika histeresis
if (suhu > upperThreshold) {
    // Suhu melampaui ambang atas → nyalakan LED
    msg.payload = {"led": 1};
    flow.set("lastLedState", 1);
    return msg;
} else if (suhu < lowerThreshold) {
    // Suhu turun di bawah ambang bawah → padamkan LED
    msg.payload = {"led": 0};
    flow.set("lastLedState", 0);
    return msg;
} else {
    // Suhu berada di zona deadband → JANGAN kirim pesan apapun
    // LED mempertahankan status terakhirnya
    return null;
}
```

Penjelasan Mekanisme Kode:

1. `flow.get("lastLedState")` dan `flow.set("lastLedState", ...)`: Menggunakan *flow context* sebagai memori persisten antar-pesan. Variabel ini menyimpan status LED terakhir sehingga sistem mengetahui apakah LED sedang menyala atau padam.
2. `return null`: Ketika suhu berada di zona deadband ($30.0°C \le T \le 31.5°C$30.0°C \le T \le 31.5°C), function node mengembalikan `null` yang berarti tidak ada pesan yang diteruskan ke node `mqtt out`. Ini mencegah pengiriman instruksi kendali yang tidak perlu dan menjaga status LED tetap stabil.
3. Zona deadband selebar $1.5°C$1.5°C (31.5−30.0=1.531.5 - 30.0 = 1.531.5−30.0=1.5): Memberikan jarak aman yang cukup sehingga fluktuasi sensor normal tidak memicu pergantian status LED.

Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):
Pada berkas `docker-compose.yml`, terdapat pemetaan direktori `- ./nodered:/data` dan `- ./mosquitto/data:/mosquitto/data`.

1. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah `docker compose down`, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari `docker-compose.yml`?

Jika baris `- ./nodered:/data` dihapus dari `docker-compose.yml`, maka seluruh alur kerja (flow) Node-RED yang telah dirancang akan hilang secara permanen setelah kontainer dihentikan dan dibuat ulang.

Berikut kronologi peristiwa yang terjadi:

1. Kondisi awal (volume masih ada):
   Semua data Node-RED (termasuk berkas `flows.json` yang menyimpan seluruh konfigurasi kanvas, node, koneksi, dan pengaturan broker) tersimpan di dua tempat secara sinkron:
   - Di dalam kontainer: direktori `/data/`
   - Di laptop host: direktori `./nodered/`
2. Setelah baris volume dihapus dan `docker compose down` dijalankan:
   Perintah `docker compose down` menghapus kontainer beserta seluruh *filesystem layer* yang melekat padanya. Data di dalam direktori `/data/` kontainer ikut hancur bersama kontainer.
3. Ketika `docker compose up -d` dijalankan kembali (tanpa volume):
   Docker membuat kontainer Node-RED baru dari nol (*fresh instance*) berdasarkan image `nodered/node-red:latest`. Kontainer baru ini memiliki direktori `/data/` yang kosong (hanya berisi konfigurasi *default* bawaan image). Karena tidak ada pemetaan volume ke folder `./nodered/` di laptop, Node-RED tidak memiliki akses ke berkas `flows.json` lama yang masih tersimpan di host.
4. Hasil akhir:
   Membuka `http://localhost:1880` akan menampilkan kanvas Node-RED yang kosong — semua alur telemetri, node `switch`, node `change`, konfigurasi broker `Mosquitto Local`, dan flow rule engine harus dibuat ulang dari awal.

2. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

4. Jawaban:

Aspek	Bind Mount	Named Volume
Definisi	Pemetaan langsung dari path absolut/relatif di filesystem host ke path di dalam kontainer	Volume terkelola oleh Docker yang disimpan di area penyimpanan internal Docker Engine

Aspek	Bind Mount	Named Volume
Sintaks di Compose	`- ./mosquitto/config:/mosquitto/config`	`- mosquitto_data:/mosquitto/data` (dengan deklarasi di bagian `volumes:` tingkat atas)
Lokasi penyimpanan di host	Ditentukan pengguna secara eksplisit (misalnya `./nodered/`, `D:/iot-server/mosquitto/`)	Dikelola Docker secara otomatis di direktori internal Docker (Linux: `/var/lib/docker/volumes/nama_volume/_data`)
Akses langsung dari host	☑ Mudah — pengguna dapat langsung membuka, mengedit, dan mengelola berkas melalui File Explorer atau editor teks	⚠ Sulit — lokasi fisik tersembunyi di dalam struktur internal Docker; memerlukan perintah `docker volume inspect` untuk mengetahui path-nya
Portabilitas	✗ Bergantung pada struktur direktori host; jika dipindah ke komputer lain, path mungkin tidak valid	☑ Docker mengelola siklus hidup volume secara independen; mudah dibackup dengan `docker volume` commands
Keamanan	⚠ Kontainer mendapat akses langsung ke filesystem host; potensi risiko jika salah mapping	☑ Lebih terisolasi karena dikelola dalam sandbox Docker
Perilaku saat kontainer dihapus	Data tetap ada di folder host	Data tetap ada di named volume (dihapus manual dengan `docker volume rm`)
Penggunaan ideal	Berkas konfigurasi yang sering diedit manual (seperti `mosquitto.conf`, `docker-compose.yml`)	Data aplikasi yang tidak perlu diakses langsung (database, log, cache internal)